

1차 프로젝트 기획서

2025년 12월 22일 월요일 오후 4:39

프로젝트 개요

프로젝트명

온누리상품권 가맹점 검색 & 분석 챗봇

프로젝트 기간

2024.12.22 (월) ~ 2024.12.31 (수) (6일)

프로젝트 유형

개인 프로젝트

프로젝트 목표

1. 핵심 목표

- 온누리상품권 가맹점 정보를 자연어로 검색할 수 있는 대화형 챗봇 개발
- RAG(Retrieval-Augmented Generation) 기반 정보 검색 시스템 구축
- 공공데이터를 활용한 실용적인 서비스 제공

2. 기술적 목표

- 대규모 데이터(15만+ 레코드) 전처리 및 파이프라인 구축
- 벡터 데이터베이스를 활용한 의미 기반 검색 구현
- LLM과 구조화 데이터 검색의 하이브리드 시스템 설계

3. 학습 목표

- LangChain을 활용한 RAG 시스템 실무 경험
- 공공데이터 전처리 및 활용 능력 향상
- 데이터 엔지니어링 역량 강화

프로젝트 배경 및 필요성

문제 정의

- 정보 접근성: 온누리상품권 가맹점 정보가 분산되어 있어 찾기 어려움
- 검색의 한계: 단순 키워드 검색으로는 사용자 의도 파악 어려움
- 데이터 활용: 15만+ 가맹점 데이터와 지역별 통계 데이터가 유기적으로 연결되지 않음

기대 효과

- 사용자: 자연어로 쉽게 가맹점 검색 가능

- 2. **소상공인**: 가맹점 정보 노출 증가
- 3. **정책 분석**: 지역별 판매/회수 현황 분석을 통한 인사이트 제공

데이터 소개

사용 데이터

□주요 기능

- A. 지역 기반 검색
 - B. 업종 기반 검색
 - C. 지역 + 업종 조합
 - D. 요약·통계 질문
 - E. 통계 및 분석
- 등등

시스템 아키텍처(전체 구조, 데이터 파이프라인)

사용자 질문



Streamlit UI



질문 유형 분석



└─ 정형 질문 → Pandas 검색 (지역, 업종, 시장명)



구조화 데이터 필터링



└─ 자연어 질문 → ChromaDB 벡터 검색



의미 기반 유사도 검색



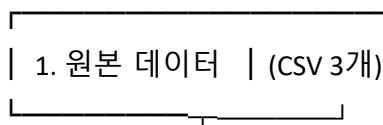
결과 통합



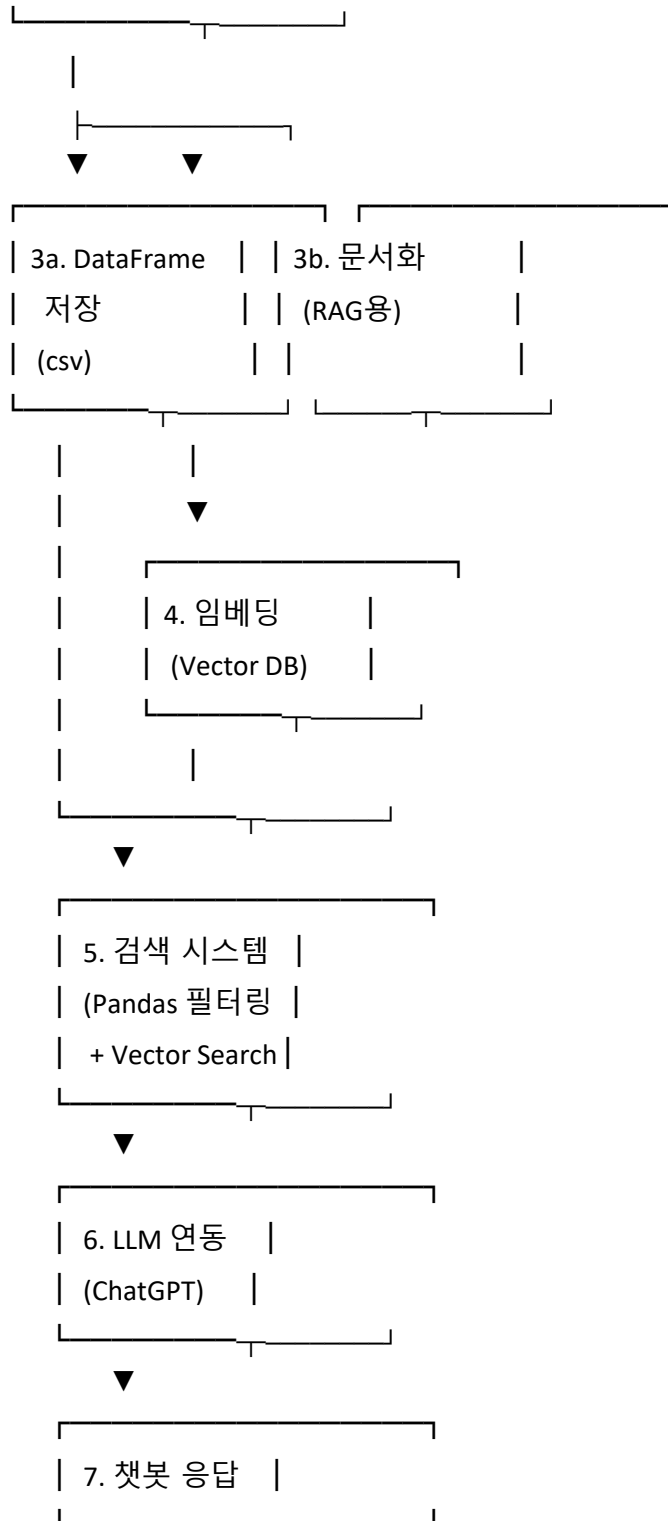
LLM (GPT-3.5) 답변 생성



사용자에게 응답



| 2. 데이터 전처리 | (결측치, 정제, 카테고리화)



기술 스택

- Python 3.11+
- LangChain (RAG 구현)
- OpenAI API (GPT-3.5-turbo)
- ChromaDB or FAISS (Vector DB)
- Pandas (데이터 처리)

개발 일정

DAY 1 (12/22, 월): 기획 + 환경 설정

오전 :

- 요구사항 정리
- 필수 4개 질문-답변 시나리오 작성
- 추가 기능 우선순위 정리

오후 :

- 개발 환경 세팅 (Python, LangChain, Streamlit)
- 데이터 다운로드 및 확인
- GitHub repo 생성

저녁 :

- 기술 스택 최종 결정
- 프로젝트 구조 설계

DAY 2 (12/23, 화): 데이터 전처리 + 저장

오전 :

- CSV 데이터 로드 및 확인
- 결측치 처리 (가맹점명 40개, 취급품목 293개)
- 데이터 정제 (공백 제거, 형식 통일)
- 정제된 DataFrame 저장

오후 :

- 데이터1 (가맹점 15만개) 적재
- 데이터2 (지역별 통계) 적재
- 지역검색 함수, 업종검색 함수, 복합 검색 함수, 통계

DAY 3 (12/24, 수): RAG 시스템 구축

오전 :

- 가맹점 정보 문서화 (가맹점명 + 소재지 + 취급품목)
- 텍스트 전처리 및 청킹

오후 :

- 임베딩 모델 선택 (OpenAI or HuggingFace)
- Vector DB 구축 (ChromaDB or FAISS)
- 15만개 가맹점 임베딩 생성 및 저장

저녁 :

- 검색 테스트 ("경기도 음식점" 검색)
- ☑체크포인트: RAG 기본 작동 확인

DAY 4 (12/26, 금): 챗봇 로직 구현 ★

오전 :

- LLM API 연동 (OpenAI GPT-3.5/4)
- 프롬프트 엔지니어링 (시스템 프롬프트 작성)

오후 :

- 필수 4개 질문 처리 로직 구현
- 지역 필터 질문 처리
- 취급품목 필터 질문 처리
- 디지털형 필터 질문 처리
- 일반 정보 질문 처리
- Day 4 끝나고 필수 기능 완성했는데 시간 남으면:
- Pandas → SQLite 마이그레이션
- 간단한 인덱싱 추가
- SQL 쿼리로 변환

저녁 :

- 각 질문 테스트 및 디버깅
- ☑핵심 체크포인트: 필수 4개 완성 ☑

DAY 5 (12/27, 토): UI 구현 + 통합

오전 :

- Streamlit 기본 레이아웃 구현
- 채팅 인터페이스 UI
- 입력창 + 대화 히스토리

오후 :

- 백엔드(RAG + LLM) ↔ 프론트엔드(Streamlit) 연결
- 전체 플로우 통합 테스트
- 필수 4개 질문 실제 UI에서 테스트

저녁 :

- 시간 여유 있으면:
- 질문 5번 추가 (지역별 가맹점 수 순위)
- 간단한 차트 추가 (matplotlib)
- 시간 부족하면:
- UI 디버깅만

DAY 6 (12/28, 일): 마무리 + 문서화

오전 :

- 전체 테스트 (다양한 질문 입력)
- 버그 수정, 오타 케이스 처리
- 에러 메시지 개선

오후 :

- 시간 여유 있으면:
- 질문 6번 추가 (2024년 회수금액 순위 - 데이터2)
- 질문 7번 추가 (지역 비교)
- 필수:

- README.md 작성
- 프로젝트 소개
- 설치 방법
- 실행 방법
- 주요 기능
- 기술 스택
- 시연 영상 녹화 또는 스크린샷

저녁 :

- 코드 정리 및 주석 추가
- GitHub 푸시
 - 코드 정리 및 주석 추가
 - GitHub 푸시

프로젝트 구조

```
onnuri-chatbot/
|
|
├── notebooks/                # 개발 노트북
|   ├── 01_data_exploration.ipynb  # 데이터 탐색
|   ├── 02_preprocessing.ipynb     # 전처리
|   ├── 03_search_engine.ipynb     # 검색 엔진
|   ├── 04_rag_setup.ipynb         # RAG 구축
|   └── 05_chatbot_test.ipynb      # 챗봇 테스트
|
├── vectordb/
|   └── chroma_db/              # 벡터 DB (자동 생성)
|
├── app.py                    # Streamlit 메인 앱
├── requirements.txt          # 패키지 목록
├── .env                      # 환경 변수 (API 키)
├── .gitignore               # Git 제외 파일
├── 가맹점_현황.csv          # 원본 데이터
├── 회수_현황.csv
├── 판매_현황.csv
├── cleaned_onnuri.csv       # 전처리된 가맹점 데이터
└── README.md                # 프로젝트 문서
```

제약사항 및 한계

1. 데이터 한계

- **소재지 정보:** 시/도 단위만 제공 (구/군 정보 없음)
- **대안:** 시장명 기반 검색 제공, 카카오 API기반 구단위 검색

2. 검색 범위

- **지원:** 시/도 단위, 업종, 시장명
- **미지원:** 구/군 단위 (예: "서울 강남구")

향후 개선 방안

입력과 출력 데이터 출처와 어떻게 출력했는지 적기

1. 상세 주소 데이터 확보 → 구/군 단위 검색
2. 실시간 영업시간 정보 추가
3. 리뷰/평점 데이터 통합
4. 모바일 앱 개발
5. 음성 인터페이스 추가
6. 개인화 추천 시스템
7. PostgreSQL 마이그레이션 (확장성)

학습 및 성장

기술 역량

- 공공데이터 전처리 경험
- RAG 시스템 설계 및 구현
- 벡터 데이터베이스 활용
- LLM 프롬프트 엔지니어링
- 데이터 파이프라인 구축

프로젝트 관리

- 기획안 작성
- 일정 관리 (6일 단위)
- Git을 활용한 버전 관리
- 기술 문서 작성

포트폴리오

- 데이터 엔지니어 역량 강화
- 실무 수준의 프로젝트 경험
- 협업 가능한 코드 작성 능력

참고 자료

공공데이터

- [공공데이터포털](#)

기술 문서

- [LangChain Documentation](#)
- [ChromaDB Documentation](#)
- [OpenAI API Reference](#)
- [Streamlit Documentation](#)